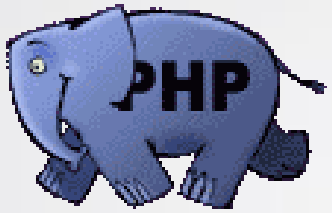
esprit
Se former autrement
HONORIS UNITED UNIVERSITIES



Chapitre 5: PHP

UP Web

AU: 2025/2026

United Nations Educational, Scientific and Cultural Organization · UNESCO Chair "Project-based learning" ESPRIT School of engineering, Tunisia

EUR-ACE® Délivrée par la Commission des Titres d'Ingénieur

CONFÉRENCE DES GRANDES ÉCOLES

cdio CONCEIVE DESIGN IMPLEMENT OPERATE

Plan

- PDO: Introduction
- PDO: Instanciation
- PDO: Exécution de Requête
- PDO: CRUD

Objectifs

- Les architectures du web
- Comprendre la syntaxe PHP
- Appréhender les notions de l'orientée objet
- Se connecter à une BD
- Manipuler les données d'une BD via PHP

Prérequis

- Langage HTML

Manipulation de la BD

PDO: PHP Data Objects

- PDO définit une interface pour accéder à une BD depuis PHP.
- PDO supporte plusieurs systèmes de bases de données:
  - MySQL
  - ODBC
  - SQLITE
  - OCI Oracle Call Interface
  - SQLite
  - …

Manipulation de la BD

PHP connexion à la BD

- Pour se connecter au serveur, on peut utiliser le fichier '**connection.php**':

```php
<?php
class config
{   private static $pdo = null;
    public static function getConnexion()
    {
        if (!isset(self::$pdo)) {
            $servername="localhost";
            $username="root";
            $password ="password";
            $dbname="myDBName";
            try {
                self::$pdo = new PDO("mysql:host=$servername;dbname=$dbname",
                        $username,
                        $password,
                    [
                        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
                        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC
                    ]
                );
                echo "connected successfully";
            } catch (Exception $e) {
                die('Erreur: ' . $e->getMessage());
            }
        }
        return self::$pdo;
    }
}
config::getConnexion();
?>
```

Manipulation de la BD

PHP connexion à la BD

Cette configuration permet de lancer une exception en cas d'erreur.

```php
<?php
class config
{   private static $pdo = null;
    public static function getConnexion()
    {
        if (!isset(self::$pdo)) {
            $servername="localhost";
            $username="root";
            $password ="password";
            $dbname="myDBName";
            try {
                self::$pdo = new PDO("mysql:host=$servername;dbname=$dbname",
                        $username,
                        $password,
                    [
                        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
                        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC
                    ]
                );
                echo "connected successfully";
            } catch (Exception $e) {
                die('Erreur: ' . $e->getMessage());
            }
        }
        return self::$pdo;
    }
}
config::getConnexion();
?>
```

Manipulation de la BD

PHP connexion à la BD

Cette configuration permet de spécifier la méthode de récupération. Dans se cas, chaque ligne est retournées dans un tableau indexé par le nom des colonnes.

```php
<?php
class config
{   private static $pdo = null;
    public static function getConnexion()
    {
        if (!isset(self::$pdo)) {
            $servername="localhost";
            $username="root";
            $password ="password";
            $dbname="myDBName";
            try {
                self::$pdo = new PDO("mysql:host=$servername;dbname=$dbname",
                        $username,
                        $password,
                    [
                        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
                        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC
                    ]
                );
                echo "connected successfully";
            } catch (Exception $e) {
                die('Erreur: ' . $e->getMessage());
            }
        }
        return self::$pdo;
    }
}
config::getConnexion();
?>
```

Manipulation de la BD

PHP connexion à la BD

- La première étape consiste à créer un objet PDO:

```
self::$pdo = new PDO("mysql:host=$servername;dbname=$dbname",$username,$password)
```

1 2 3

1. **dsn ou Data Source Name**: représente les informations nécessaires pour se connecter à la base.
2. **Username** (optionnel pour certains pilote tel que sqlite)
3. **Password**
4. **Options**: un tableau contenant les options spécifiques à la connexion. Ce paramètre est optionnel.

Remarque: La base de données (myDB) peut être créer en utilisant l'application phpMyAdmin.

Manipulation de la BD

PHP connexion à la BD

- La connexion sera fermée automatiquement lorsque le script se termine. Pour fermer la connexion avant, utilisez la commande suivante:

```
self::$pdo = null;
```

- Comment je peux inclure le fichier de « **connection.php** »?

Récrire le code connection.php, utiliser les fonctions :

- require ()
- require-once()
- include()
- include-once()

Manipulation de la BD

PHP connexion à la BD

- La différence entre include et include_once:
  - include_once inclut et évalue le fichier spécifié durant l'exécution du script. Le comportement est similaire à include, mais la différence est que si le code a déjà été inclus, il ne le sera pas une seconde fois, et include_once retourne TRUE.
  - La structure *include_once* est utilisée:
    - de préférence lorsque le fichier va être inclus ou évalué plusieurs fois dans un script,
    - ou bien lorsque vous voulez être sûr qu'il ne sera inclus qu'une seule fois, pour éviter des redéfinitions de fonctions ou de classes.

Manipulation de la BD

PHP connexion à la BD

- La différence entre include et require:
  - require est identique à include mise à part le fait que lorsqu'une erreur survient, il produit également une erreur fatale de niveau E_COMPILE_ERROR.
  - Il stoppera le script alors que include n'émettra qu'une alerte de niveau E_WARNING, ce qui permet au script de continuer.

Manipulation de la BD

PHP connexion à la BD

- La différence entre require et require_once?
  - L'instruction *require_once* est identique à require mis à part que PHP vérifie si le fichier a déjà été inclus, et si c'est le cas, ne l'inclut pas une deuxième fois.

=> Pour faire appel au fichier « config.php » il suffit:

```
<?php

require '../config.php';
//.............................



?>
```

Manipulation de la BD

PDO: exec(), query(), execute(), prepare()

Fonction	Explication
PDO::exec(string $sql): int	Permet d'exécuter une requête. Renvoie le nombre de ligne affectées. À utiliser avec: Insert, Update et Delete.
PDO::query(string $sql): PDOStatement	Exécute une requête. Retourne le résultat en un objet PDOStatement
PDO::prepare(string $sql): PDOStatement	Prépare la requête sans l'exécuter. Renvoie un objet PDOStatement.
PDOStatement::execute([array $param]): bool	Exécute une requête préparée.

Manipulation de la BD

PDO: CRUD

CRUD: ‘Create’ des données

L'instruction INSERT INTO est utilisée pour ajouter de nouveaux enregistrements à une table.

```php
function addJoueur($joueur)
{
    $sql = "INSERT INTO joueur (id, nom, prenom, email, tel)
            VALUES (NULL, :nom, :prenom, :email, :tel)";
    $db = config::getConnexion();
    try {
        $query = $db->prepare($sql);
        $query->execute([
            'nom' => $joueur->getNom(),
            'prenom' => $joueur->getPrenom(),
            'email' => $joueur->getEmail(),
            'tel' => $joueur->getTel(),
        ]);
    } catch (Exception $e) {
        echo 'Error: ' . $e->getMessage();
    }
}
```

Manipulation de la BD

PDO: CRUD

CRUD: ‘Create’ des données

L'instruction INSERT INTO est utilisée pour ajouter de nouveaux enregistrements à une table.

PDOStatement::bindValue(
$param,
$value,
[$data_type]
) : bool

```
function addJoueur($joueur)
{
    $sql = "INSERT INTO joueur (id, nom, prenom, email, tel)
            VALUES (NULL, :nom, :prenom, :email, :tel)";
    $db = config::getConnexion();
    try {
        $query = $db->prepare($sql);


        $query->bindValue(':nom', $joueur->getNom());
        $query->bindValue(':prenom', $joueur->getPrenom());
        $query->bindValue(':email', $joueur->getEmail());
        $query->bindValue(':tel', $joueur->getTel());

        $query->execute();
    } catch (Exception $e) {
        echo 'Error: ' . $e->getMessage();
    }
}
```

Manipulation de la BD

PDO: CRUD

CRUD: ‘Create’ des données

L'instruction INSERT INTO est utilisée pour ajouter de nouveaux enregistrements à une table.

PDOStatement::bindParam
($param,
$value,
[$data_type],
[$length]
) : bool

```
function addJoueur($joueur)
{

    $sql = "INSERT INTO joueur (id, nom, prenom, email, tel)
            VALUES (NULL, :nom, :prenom, :email, :tel)";
    $db = config::getConnexion();

    try {
        $query = $db->prepare($sql);
        $query->bindParam(':nom', $joueur->getNom());
        $query->bindParam(':prenom', $joueur->getPrenom());
        $query->bindParam(':email', $joueur->getEmail());
        $query->bindParam(':tel', $joueur->getTel());

        $query->execute();
    } catch (Exception $e) {

        echo 'Error: ' . $e->getMessage();
    }
}
```

Manipulation de la BD

PDO: CRUD

CRUD: ‘Read’ des données
L'instruction SELECT est utilisée pour sélectionner les données à partir d'une table.

```
function getJoueurs()
{
    $sql = "SELECT * FROM joueur";
    $db = config::getConnexion();

    try {

        $query = $db->prepare($sql);
        $query->execute();
        return $query->fetchAll();
    } catch (Exception $e) {
        echo 'Error: ' . $e->getMessage();
    }
}
```

Manipulation de la BD

PDO: CRUD

CRUD: ‘Update’ des données
L'instruction UPDATE est utilisée pour modifier les données d'une table.

```php
function updateJoueur($joueur, $id)
{
    try {
        $db = config::getConnexion();
        $query = $db->prepare(
            'UPDATE joueur SET
                nom = :nom,
                prenom = :prenom,
                email = :email,
                tel = :tel
            WHERE idJoueur = :id'
        );
        $query->execute([
            'id' => $id,
            'nom' => $joueur->getNom(),
            'prenom' => $joueur->getPrenom(),
            'email' => $joueur->getEmail(),
            'tel' => $joueur->getTel(),
        ]);

        echo $query->rowCount() . " records UPDATED successfully <br>";
    } catch (PDOException $e) {
        $e->getMessage();
    }
}
```

Manipulation de la BD

PDO: CRUD

CRUD: ‘Delete’ des données
L'instruction DELETE est utilisée pour supprimer des données d'une table.

```
function deleteJoueur($id)
{
    try {

        $db = config::getConnexion();
        $query = $db->prepare('DELETE FROM joueur WHERE idJoueur = :id');
        $query->execute([
            'id' => $id
        ]);
        echo $query->rowCount() . " record(s) DELETED successfully <br>";
    } catch (PDOException $e) {

        echo 'Error: ' . $e->getMessage();
    }
}
```

Merci de votre attention